Introduction to Module 3: Recursion

Design and Analysis of Algorithms

Brian C. Dean

School of Computing
Clemson University

Recursion: A Powerful Idea

- Useful as design technique, especially for problems with “recursive substructure” (large problem solvable by reducing to smaller subproblems).
  - E.g., Divide and conquer
  - E.g., Dynamic programming in optimization
  - E.g., Figure out small piece of solution; delegate rest to recursion.
- Often leads to simple, concise, and elegant implementation in code.

Recursion: A Powerful Idea

- Useful as design technique, especially for problems with "recursive substructure" (large problem solvable by reducing to smaller subproblems).
  - E.g., Divide and conquer
  - E.g., Dynamic programming in optimization
  - E.g., Figure out small piece of solution; delegate rest to recursion.

<u>Reduce stress by living your life by recursion:</u>
Only worry about the next 5 minutes, then
Recursively apply self to rest of life.

Recursion Related to Mathematical Induction

- Prove: $4^N - 1$ is a multiple of 3.

  Easy: $4^N - 1 = \underbrace{4^{N-1} + 4^{N-1} + 4^{N-1}}_{\text{3 times something}} + \underbrace{4^{N-1} - 1}_{\text{a multiple of 3 by induction}}$

- When proving statement for N, can assume it holds for values smaller than N.
- When arguing correctness of an algorithm applied to input of size N, can assume it works on inputs smaller than N.

In This Module…

- Emphasis on “divide and conquer” methods.
- How to analyze them by solving recurrences.
- Many examples of such algorithms – e.g., selection.
- Coding discussion on recursive thinking and implementation.
- Optional enrichment lecture: FFTs.
- Fun assignment on “fair division”.

- In later modules: dynamic programming and greedy algorithms for solving discrete optimization problems with recursive substructure.